Scriptavera: Technical Overview and 20 Step Pipeline Architecture

Supplementary Technical Document for the AI for Math Fund Seed Grant

Applicant and Solo Founder: Vaibhav Chhajer (vchhajer0504@gmail.com)

2 Minute Video Walkthrough: https://youtu.be/BoExpXqJ0BY

Open Source Repository (AGPL 3.0): https://github.com/Chhajer2000/scriptavera

1. Problem Statement and Project Summary

Mathematicians write proofs on paper, yet structuring, shortening, polishing, and rechecking raw handwritten notes into a clean, publication ready LaTeX paper takes weeks of tedious work that produces no new mathematical results. Standard AI chat tools cannot be trusted for this work because a single unconstrained prompt silently changes notation, drops crucial assumptions, skips case splits, hallucinates non existent citations, or invents false proof steps.

Scriptavera (*scripta vera*, meaning "true writings") is an open source **20 step pipeline** written in over 15,000 lines of Python and JavaScript that turns handwritten research notes (scanned PDFs or photos) into a complete LaTeX research paper. Each AI call performs one focused job, every step saves its state on disk, the system stops at **6 mandatory author checkpoints** to ask the human mathematician decisions only the author can make, and a deterministic **mechanical formula auditor** verifies every equation symbol by symbol.

Open Source Core and Closed Source Commercial Model: This 20 step pipeline is completely free in open source under the AGPL 3.0 license so any researcher or university can run it locally with their own API keys. Alongside this free open source project, I will also build my own closed source commercial version with managed cloud compute and zero setup execution for long term sustainability.

2. How Scriptavera Protects Mathematical Integrity

- **Mechanical Formula Auditor (`audit.py`):** Section writers may improve English wording, but they are forbidden from changing formulas. After drafting and copy editing, Scriptavera mechanically compares every LaTeX formula in the paper symbol by symbol against the author's approved transcript. Missing or altered formulas are automatically flagged and restored. A copy edit is kept only if every formula, label, and citation remains untouched.
- **Six Author Checkpoints (Mathematical Questions Only):** The pipeline stops up to 6 times to ask the author strictly about mathematics: confirming the problem and literature, defining ambiguous symbols, reviewing page transcripts and TikZ figures, deciding how to frame incomplete proofs, approving or rejecting proposed proof simplifications or corrections, and selecting the final title, introduction, and citations. The author is never asked about spelling or grammar.
- **Literature Verification Across 5 Databases:** Every cited paper is searched and verified across **arXiv, zbMATH, OpenAlex, Crossref, and Semantic Scholar**, with direct lookup links to Google Scholar, preventing fabricated bibliography entries.
- **Honest Assessment of Unfinished Notes:** Before drafting, Scriptavera checks whether each result in the notes is fully proved, has a gap, is only sketched, or already appears in the read literature, ensuring unproved steps are stated honestly as conjectures or conditional claims rather than proved theorems.

3. The Complete 20 Step Pipeline and 6 Author Checkpoints

Phase 1: Reading Your Handwritten Notes (Steps 1 to 9)

1. **Step 1: Scout Read.** Reads a few pages (words only, equations skipped) to identify the mathematical area and topic. Blank pages or notebook covers with no text, formulas, or drawings are safely left out and shown to the author.
2. **Step 2: Literature Search.** Searches for relevant papers in the author's area across arXiv, zbMATH, OpenAlex, Crossref, and Semantic Scholar, starting with any foundational papers the author specifies.
3. **Step 3: Author Checkpoint 1 (Your Topic).** You see what the app understood and the papers it found. You can confirm, exclude papers, or refine the problem and keywords; if you comment, the topic is rewritten

and the search repeats until you confirm.

4. **Step 4: Notation Survey.** Reads every page with the confirmed topic known and catalogues every symbol and special term across the notes.
5. **Step 5: Author Checkpoint 2 (Your Confirmation).** You confirm the topic and clarify every symbol or term the notes use without defining. A page button marks on the handwritten scan exactly where the symbol was read.
6. **Step 6: Deep Reading.** Runs a refined search and reads the most relevant papers one by one (full text when open access, otherwise the abstract).
7. **Step 7: Final Transcription.** Reads every handwritten page again using the confirmed notation ledger and research context to produce an accurate LaTeX transcript.
8. **Step 8: Figures.** Redraws each handwritten sketch as a compilable TikZ figure. Labels written on a formula with arrows or braces are converted into LaTeX `\underbrace` annotations inside the formula rather than separate figures.
9. **Step 9: Author Checkpoint 3 (Your Review).** You check the LaTeX transcript side by side against your page scans and approve, redraw, or replace each TikZ figure.

Phase 2: First Draft and Structural Understanding (Steps 10 to 14)

10. **Step 10: Understanding the Paper.** Reads all notes together with the literature to map the problem, every result, and the dependency graph of what depends on what. A second independent reading verifies that every extracted result is genuinely stated as a standalone claim and not an internal step or case of a longer proof that would lose its assumptions.
11. **Step 11: Assessment of the Notes.** Evaluates every result: whether it is fully proved in the notes, has a gap, is only sketched, or already appears in the read papers, and whether the main theorem stands on what is proved.
12. **Step 12: Author Checkpoint 4 (Your Plan for the Paper).** You decide whether the paper is a full research article, a paper presenting unproved steps as conjectures or conditional results, or an expository note. Passed automatically if every result is proved and novel.
13. **Step 13: Introduction.** Writes a working introduction used by the section writers as context; you approve the final introduction at the end.
14. **Step 14: Structure and First Draft.** Decides the order of definitions, lemmas, and theorems so multi page arguments stay in one section. Each section writer sees the whole notes and writes its assigned pages. Every formula of your notes is compared mechanically (`audit.py`) with the draft, and missing or altered formulas are restored.

Phase 3: Revision, Proof Check, and Refereeing (Steps 15 to 18)

15. **Step 15: Shorter Arguments.** Reads the draft again for arguments carried out twice, methods that can be stated once as a general lemma, and roundabout proofs that can be shortened. Each proposal is verified by a second independent reading.
16. **Step 16: Proof Check.** Examines every proof step by step for logical gaps or hidden assumptions. Each suspected problem goes to an independent second opinion, and only issues upheld by both readings are reported.
17. **Step 17: Referee's Read.** Reads the whole manuscript the way a strict journal referee evaluates a submission.
18. **Step 18: Author Checkpoint 5 (Your Decisions).** Anything that touches the mathematics (a shortened proof, a logical correction, or a mathematical remark) is applied only if you explicitly approve it.

Phase 4: Final Version and Journal Formatting (Steps 19 to 20)

19. **Step 19: Final Version.** Carries out your decisions from Step 18, copy edits prose once (kept only if formulas and citations are untouched), corrects mathematical typography by rule, formats for your target journal (`amsart`, `article`, or `elsarticle`), writes the abstract and five title suggestions, and compiles `main.tex`, `references.bib`, the PDF, and an Overleaf zip package.
20. **Step 20: Author Checkpoint 6 (Title, Introduction, Citations).** You choose the title, approve or edit the introduction, tick which verified papers to cite, and review a readiness checklist showing anything still standing between the manuscript and journal submission.

4. Key Seed Grant Objectives

1. **Multi Model Routing for Proof Review and Paper Writing:** Connect Scriptavera's 6 specialized roles to the best models for each task: (1) vision models for reading messy handwritten math and diagrams (Steps 1, 4, 7, 8), (2) long context models for literature search and synthesis across arXiv, zbMATH, OpenAlex, Crossref, and Semantic Scholar (Steps 2, 6), (3) structural extraction models for building the theorem dependency graph (Steps 10, 11), (4) prose drafting models guarded by our mechanical formula auditor (Steps 13, 14, 19), (5) mathematical reasoning models for checking proof logic step by step and asking the author before changing any step (Steps 15, 16), and (6) adversarial referee models for journal review (Step 17).
2. **Priority Timestamping for Unpublished Notes:** Build local SHA 256 hashing and RFC 3161 trusted timestamp certificates immediately upon note upload, giving authors legal proof of priority before any external API call is made.
3. **Lean 4 Theorem Stub Export:** Convert the paper's theorem dependency graph (Step 10) into compilable Lean 4 theorem skeletons (`:= by sorry`) to bridge handwritten mathematical research with formal verification.
4. **Expanding Beyond Mathematics to All Scientific Fields Later On:** Initially in Year 1 we are focused 100% on mathematics; once mathematics is proven, we will expand the pipeline in Year 2 to physics, computer science, and all other subjects.

5. Implementation Plan and Key Milestones

- **Baseline Built Today:** Scriptavera v1.0 (15,000+ lines of Python and JS, AGPL 3.0, https://github.com/Chhajer2000/scriptavera | Video: https://youtu.be/BoExpXqJ0BY) is live with all 20 pipeline steps, 6 author checkpoints, 5 database citation verification, and the mechanical formula auditor.
- **Milestone 1 (Months 1 to 4):** Build Step 0 SHA 256 and RFC 3161 priority timestamping, 6 role model routing, Step 16 interactive proof review, and Lean 4 theorem skeleton export from Step 10.
- **Milestone 2 (Months 5 to 8):** Further improve accuracy across all 20 steps through smart prompt engineering and strengthen the overall software engineering aspects, speed, state recovery, and LaTeX reliability of the project as a whole.
- **Milestone 3 (Months 9 to 12):** Distribute the system to university mathematics institutes for free to test the project on real research papers, gather researcher feedback, and identify where to improve.
- **Milestone 4 (Year 2, Requiring at Least 1 Full Year):** Expanding to other subjects cannot be rushed in a few months; once mathematics is proven in Year 1, we will dedicate Year 2 (at least 12 months) to expanding the project to physics, computer science, and other subjects.

6. Team Fit

I am currently a single founder working alone without a team, and having faced the pain of turning handwritten proofs into LaTeX papers myself, I built all 15,000+ lines of Scriptavera v1.0 alone. I hold an **MSc in Mathematics from Central University of Karnataka**, qualified the national **GATE 2024 examination in Mathematics with All India Rank (AIR) 299**, and completed **1 year of mathematical research at NISER** (National Institute of Science Education and Research), during which I wrote two research papers in group theory:

1. *Finite groups with mostly involuted cyclic subgroups* (with Palash Sharma), published in **Indian Journal of Pure and Applied Mathematics** (Springer): https://doi.org/10.1007/s13226-026-01029-y (arXiv: https://arxiv.org/abs/2508.00681).
2. *Finite groups with nearly half as many cyclic subgroups as elements* (with Sumana Hatui and Palash Sharma), accepted in **Proceedings Mathematical Sciences** (Indian Academy of Sciences / Springer): https://arxiv.org/abs/2506.21163.

With this grant, I will work full time on Scriptavera and hire two full time engineers for 12 months: an **Applied AI Engineer** (multi model routing, smart prompt engineering, and Lean 4 export) and a **Full Stack Backend Engineer** (pipeline engineering, RFC 3161 timestamping, and system reliability).

7. 12 Month Budget Summary ($99,000 USD at Live Rate 1 USD = Rs 96 INR)

Category	Description (12 Month Grant Period, Converted at 1 USD = Rs 96 INR)	Amount (USD)
Personnel	**PI & Mathematical Lead** (Vaibhav Chhajer, 100% full time, Rs 2,00,000/mo, Rs 24,00,000/yr): $25,000 **Applied AI Engineer** (100% full time, Rs 2,00,000/mo, Rs 24,00,000/yr): $25,000 **Full Stack Backend Engineer** (100% full time, Rs 2,00,000/mo, Rs 24,00,000/yr): $25,000	$75,000
Equipment	3 developer laptops/PCs, monitors, and an iPad tablet with stylus for handwritten math note testing (Rs 5,28,000 INR)	$5,500
Travel	Domestic travel to university mathematics institutes in Milestone 3 to distribute the system for free and gather researcher feedback (Rs 1,92,000 INR)	$2,000
Other Direct	**GPU Server Rental & Frontier API Compute** for development, dual pass proof review, and free university institute testing ($9,000 / Rs 8,64,000 INR) **Managed 3 Seat Office Cabin & Infrastructure** for 12 months ($5,500 / Rs 5,28,000 INR) **Legal, Accounting, & RFC 3161 Timestamping Infrastructure** ($2,000 / Rs 1,92,000 INR)	$16,500
Indirect Costs	0% Overhead (100% of grant funds go directly to engineering and institutional testing)	$0
Total Requested	**Total 12 Month Seed Grant Request (Rs 95,04,000 INR at 1 USD = Rs 96 INR)**	**$99,000**